

Vertex UI Engine Specification

Version: 1.0

Status: Draft

1. Objetivo

O UI Engine é o subsistema do Vertex responsável por definir, organizar e coordenar toda a estrutura da interface da plataforma.

Seu objetivo é fornecer um modelo declarativo, consistente e independente de implementação para a construção de interfaces de plugins e componentes do sistema.

O UI Engine descreve a interface. A renderização e execução pertencem ao UI Runtime Engine.

2. Responsabilidades

O UI Engine é responsável por:

- definir a árvore de componentes;
- gerenciar navegação;
- organizar layouts;
- controlar composição da interface;
- processar estados visuais;
- coordenar animações declarativas;
- integrar entrada do usuário;
- fornecer componentes reutilizáveis.

O UI Engine não desenha pixels na tela.

3. Arquitetura

Plugin

↓

UI API

↓

UI Engine

- |— Component Tree
- |— Layout Engine
- |— Navigation Manager
- |— Composition Engine
- |— Theme Manager
- |— Animation Coordinator
- |— Input Coordinator
- |— Accessibility Manager
- |— UI Metrics

↓

UI Runtime Engine

↓

Android / Compose

4. Componentes

Component Tree

Mantém a árvore declarativa da interface.

Cada elemento possui:

- tipo;
- propriedades;
- estado;
- filhos.

Layout Engine

Calcula a organização lógica dos componentes.

Responsável por:

- alinhamento;
- dimensões;
- posicionamento;
- restrições.

Navigation Manager

Controla:

- telas;
- pilha de navegação;
- transições;
- histórico.

Composition Engine

Reconstrói partes da interface quando necessário.

Atualiza apenas componentes afetados.

Theme Manager

Gerencia:

- tema;
- cores;
- tipografia;
- dimensões;
- estilos.

Animation Coordinator

Coordena animações declarativas.

Define:

- duração;
- curvas;
- sincronização;
- cancelamento.

Input Coordinator

Recebe entradas do usuário.

Suporta:

- toque;
- gestos;
- botão físico;
- coroa rotativa;
- futuros dispositivos de entrada.

Accessibility Manager

Fornece recursos de acessibilidade.

Exemplos:

- descrição de componentes;
- foco;
- navegação assistida;
- escalabilidade visual.

UI Metrics

Coleta métricas de desempenho da interface.

5. Fluxos

Construção da Interface

Plugin

↓

Component Tree

↓

Layout Engine

↓

Composition

↓

UI Runtime

Atualização

State Change

↓

Composition Engine

↓

Layout

↓

Runtime

↓

Tela Atualizada

Entrada

Usuário

↓

Input Coordinator

↓

Componente

↓

Ação

6. APIs

Componentes

- createComponent()
- removeComponent()

Navegação

- navigate()
- back()
- replace()

Layout

- requestLayout()
- invalidate()

Tema

- currentTheme()
- observeTheme()

Entrada

- registerInput()
- unregisterInput()

Interface

- showScreen()
- closeScreen()
- refresh()

7. Estados

Uma tela pode assumir:

Created



Composed



Visible



Focused



Background



Hidden



Destroyed

Componentes individuais podem estar:

- Enabled;
- Disabled;
- Loading;
- Selected;
- Hidden.

8. Políticas

Declaratividade

Interfaces devem ser declarativas.

Reutilização

Componentes devem ser reutilizáveis.

Atualização Parcial

Somente regiões modificadas devem ser recompostas.

Responsividade

A interface deve adaptar-se automaticamente aos diferentes formatos de tela.

Eficiência

Evitar recomposições desnecessárias.

Acessibilidade

Todo componente deve oferecer suporte à acessibilidade.

Independência

Plugins descrevem interfaces.

O Core decide como renderizá-las.

9. Integrações

O UI Engine integra-se com:

- UI Runtime Engine;
- Plugin API;
- Plugin Context;
- State Engine;
- Event System;
- Scheduler;
- Asset Manager;
- Resource Manager;
- Configuration System;
- Logging Framework;

- Memory Management;
- Power Management;
- Android Bridge.

A implementação padrão utiliza Jetpack Compose, mas o restante da plataforma permanece desacoplado dessa tecnologia.

10. Métricas

O UI Engine registra:

- número de componentes;
- recomposições;
- tempo de composição;
- tempo de layout;
- navegações realizadas;
- animações executadas;
- eventos de entrada;
- consumo de memória da interface;
- taxa de atualização da interface;
- componentes reutilizados.

Essas métricas auxiliam o Plugin Inspector e ferramentas de diagnóstico.

11. Exemplos

Tela Inicial

1. Plugin cria a tela.
2. Component Tree registra os componentes.
3. Layout Engine organiza o conteúdo.
4. UI Runtime renderiza a interface.

Mudança de Tema

1. O tema do sistema é alterado.
2. Theme Manager atualiza os valores.
3. O Composition Engine recompõe apenas os componentes afetados.
4. O Runtime redesenha a interface.

Atualização por Estado

1. O State Engine altera "battery.level".
2. O componente observa esse estado.
3. Apenas o indicador de bateria é recomposto.
4. O restante da interface permanece inalterado.

Entrada pela Coroa Rotativa

1. O usuário gira a coroa do relógio.
2. O Input Coordinator recebe o evento.
3. O componente focado processa a rolagem.
4. A interface é atualizada conforme necessário.

12. Regras Arquiteturais

- O UI Engine nunca desenha diretamente na tela.
- Toda renderização pertence ao UI Runtime Engine.
- Toda interface deve ser declarativa.
- Componentes devem ser independentes e reutilizáveis.
- O estado da interface deve ser derivado do State Engine sempre que possível.
- Entradas devem ser abstraídas do hardware específico.

13. Limitações

O UI Engine não é responsável por:

- renderização gráfica;
- execução de animações em baixo nível;
- gerenciamento de memória gráfica;
- acesso direto ao Android;
- execução de plugins.

Essas funções pertencem ao UI Runtime Engine e a outros subsistemas.

14. Futuras Extensões

O UI Engine poderá evoluir para oferecer:

- layouts adaptativos avançados para diferentes formatos de dispositivos;

- componentes 3D e animações complexas;
- suporte a múltiplas janelas;
- temas dinâmicos baseados em contexto;
- componentes distribuídos entre relógio e smartphone;
- ferramentas visuais de construção de interfaces.

15. Resumo

O UI Engine define a estrutura lógica das interfaces do Vertex. Ele organiza componentes, layouts, navegação e estados visuais de forma declarativa e desacoplada da renderização. Em conjunto com o UI Runtime Engine, oferece uma base moderna, eficiente e escalável para interfaces de smartwatch, aproveitando a implementação padrão em Jetpack Compose sem criar dependência arquitetural da tecnologia subjacente.